

### 1 List [ ]

- Mutable, Ordered, Duplicates Allowed
- Access: `list[index]`
- Slicing: `list[start:stop:step]`
- Reverse: `list[::-1]` or `list.reverse()`
- Rotate:
- Left rotate by 2: `lst[2:] + lst[:2]`
- Right rotate by 2: `lst[-2:] + lst[:-2]`
- Useful Functions: `append()`, `insert()`, `remove()`, `pop()`, `sort()`, `reverse()`, `count()`, `len()`

### 2 Tuple ( )

- Immutable, Ordered, Duplicates Allowed
- Access & slicing: same as list
- Functions: `count()`, `index()`, `len()`
- Use: fixed data like coordinates, RGB, date

### 3 Set { }

- Mutable, Unordered, Unique Items Only
- No index, no slicing
- Useful Functions: `add()`, `remove()`, `discard()`, `pop()`, `union()`, `intersection()`, `difference()`, `len()`
- Use: unique items, fast membership check

### 4 Dictionary {key: value}

- Mutable, Unordered, Keys Unique
- Access: `dict[key]` or `dict.get(key)`
- Functions: `keys()`, `values()`, `items()`, `get()`, `update()`, `pop()`, `len()`
- Use: key-value mapping, frequency counting, fast lookup

### 5 Frequency / Counting

- Single Value Count (List/Tuple): `lst.count(value)`
- All Elements Frequency (Dictionary):

```
freq = {}  
for x in lst:  
    freq[x] = freq.get(x, 0) + 1
```

- Set + count Shortcut (Small List):

```
for x in set(lst):  
    print(x, lst.count(x))
```

Tip: Small lists → set + count, Large lists → dict frequency, Single element → count()

## 6 Max / Min

- `max(lst)`, `min(lst)`

## 7 Logical Conditions

```
if condition:  
    # code  
else:  
    # code
```

- Used in filtering, frequency problems, max/min, etc.

## 8 Quick Rules / Memory Tricks

- List → Row of boxes, can change
- Tuple → Fixed row, safe & fast
- Set → Bag of unique items
- Dict → Label → Value mapping
- `count()` → Single element frequency
- dict freq → All elements frequency
- set+count → Shortcut for small list

---

💡 Practice Tip: - Solve 5–10 small problems per day using this cheat sheet - Always dry run before running code → strong logic + memory